



第六章 层次结构存储系统

主要内容



华中科技大学

◆ 6.1 存储器的层次结构

◆ 6.2 高速缓冲存储器





6.1.存储器的层次结构

存储器是计算机系统中的重要组成部分。计算机中所用的存储元件有多种类型，如：

- 寄存器
- 静态RAM
- RAM
- Flash存储器
- 磁盘
- 光盘



一、存储器的分类

1.按存取方式分类

(1)随机存取存储器

(2)顺序存取存储器

(3)直接存取存储器

(4)相联存储器



1.按存取方式分类

(1)随机存取存储器

随机存取存储器(Random Access Memory,简称RAM)的特点是按地址访问存储单元,每个单元的访问时间是一个常数,与地址无关。

(2)顺序存取存储器

顺序存取存储器(Sequential Access Memory,简称SAM)的特点是信息按顺序存放和读出,其存取时间取决于信息存放位置,以记录块为单位编址。存储容量大,但存取速度慢。如磁带存储器。





1.按存取方式分类

(3)直接存取存储器

直接存取存储器(Direct Access Memory,简称DAM)的存取方式兼有随机访问和顺序访问的特点。首先直接定位到需读写信息所在区域的开始处,然后按顺序方式存取,如磁盘存储器。

(4)相联存储器

上述三类存储器都是按地址来访问。但有些情况下只知道要访问信息的内容特征,只能按内容检索到存储位置进行读写。这种存储器称为按内容访问存储器(Content Addressed Memory, CAM)或相联存储器(Associative Memory,简称AM)。如快表。



2.按信息的可更改性分类

(1)读写存储器(Read/Write Memory)

信息可以读出和写入，如RAM芯片。

(2)只读存储器(Read Only Memory)。

存储器芯片中的信息一旦确定，通常只能读不能写。

如ROM芯片。

RAM芯片和ROM芯片都采用随机存取方式进行信息的访问。



3.按断电后信息的可保存性分类

(1)非易失性存储器(Nonvolatile Memory)

也称不挥发性存储器，其信息可一直保留，不需电源维持，例如，ROM、磁表面存储器、光盘存储器等；

(2)易失性存储器(Volatile Memory)。

也称挥发性存储器，在电源关闭时信息自动丢失，例如，RAM、cache等都是易失性存储器。



4.按功能分类

- (1)高速缓冲存储器
- (2)主存储器
- (3)辅助存储器
- (4)海量后备存储器



4.按功能分类

(1)高速缓冲存储器

简称高速缓存，位于主存和CPU之间，目前主要由静态RAM芯片组成，其存取速度接近CPU的工作速度，用来存放当前CPU经常使用到的指令和数据。

(2)主存储器

指令直接面向的存储器是主存储器，简称主存。CPU执行指令时给出的存储地址最终必须转换为主存地址。主存是存储器分层结构中的核心存储器，用来存放系统中启动运行的程序及其数据，主存目前一般用MOS管半导体存储器构成。





4.按功能分类

(3)辅助存储器

把系统运行时直接和主存交换信息的存储器称为辅助存储器，简称辅存。目前大多用磁盘存储器作为辅存，辅存的内容需要调入主存后才能被CPU访问。

(4)海量后备存储器

磁带存储器和光盘存储器的容量大、速度慢，主要用于信息的备份和脱机存档，因此被用作海量后备存储器。

辅存和海量后备存储器统称为外部存储器，简称外存。





二、存储器的层次结构

通常在计算机内部采用层次化的存储器体系结构。

一种元件制造的存储器很难同时满足大容量、高速度和低成本的要求。如：

SRAM的存取速度快，但是难以构成大容量存储器

；

大容量、低成本的磁表面存储器的存取速度又远远低于半导体存储器，并且难以实现随机存取。





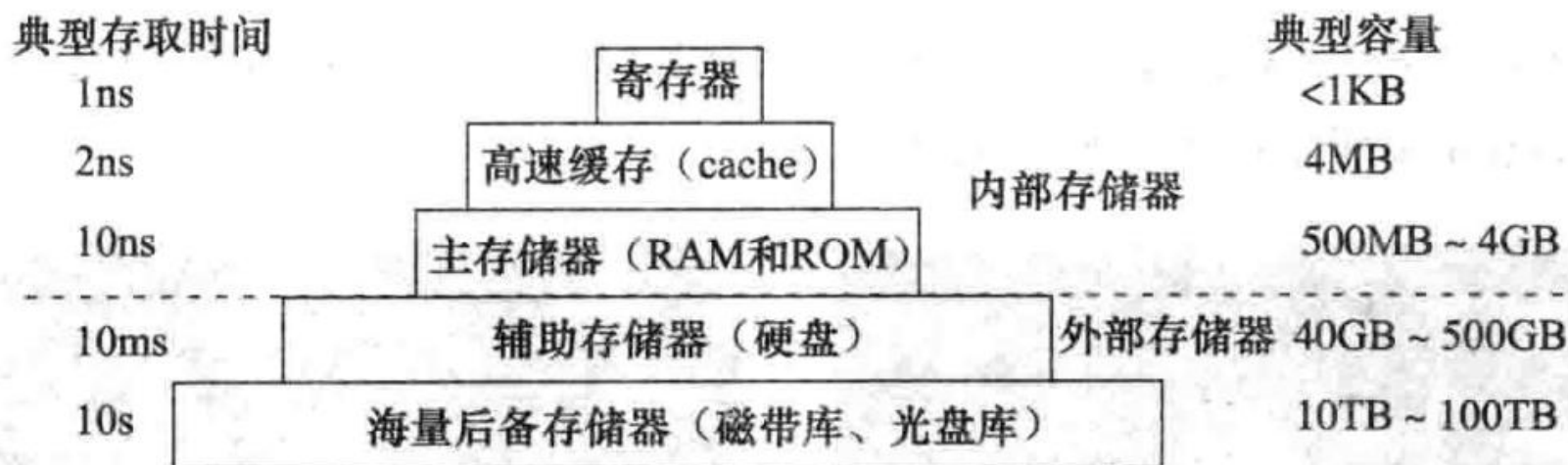
二、存储器的层次结构

为了缩小存储器和处理器两者之间在性能方面的差距，需要把各种不同的存储器按一定的结构组织在一起，形成层次化的存储器结构。

把程序和数据按不同的层次存放在各级存储器中，使得整个存储系统在速度、容量和价格等方面具有较好的综合性能指标。

二、存储器的层次结构

存储系统层次结构示意图





二、存储器的层次结构

可以看出：

- 速度越快，则容量越小，越靠近CPU
- CPU可以直接访问内部存储器，而外部存储器的信息则要先取到主存，然后才能被CPU访问
- CPU执行指令时，需要的操作数大部分都来自寄存器
- 当需要从存储器中存取数据时，先访问cache,如果不在cache中，则访问主存，如果不在主存中，则访问硬盘
- 此时，操作数从硬盘中读出送到主存，然后从主存送到cache。
- 数据使用时一般只在相邻两层之间复制传送，而且总是从慢速存储器复制到快速存储器。
- 传送的单位是一个定长块，并在相邻两层间建立块之间的映射关系。



主要内容



华中科技大学

◆ 6.1 存储器的层次结构

◆ 6.2 高速缓冲存储器





一、局部性原理

对程序运行情况分析发现，在较短时间间隔内，程序产生的地址往往集中在存储空间的一个很小范围，这种现象称为程序访问的局部性。

这种局部性可细分为时间局部性和空间局部性：

- 时间局部性是指被访问的某个存储单元在一个较短的时间间隔内很可能又被访问。
- 空间局部性是指被访问的某个存储单元的邻近单元在一个较短的时间间隔内很可能也被访问。





一、局部性原理

出现程序访问的局部性特征是因为：

程序是由指令和数据组成的，指令在主存按顺序存放，其地址连续，循环程序段或子程序段通常被重复执行，因此，指令具有明显的访问局部化特征；

数据在主存一般也是连续存放，特别是数组元素，常常被按序重复访问，因此，数据也具有明显的访问局部化特征。





一、局部性原理

出现程序访问的局部性特征是因为：

程序是由指令和数据组成的，指令在主存按顺序存放，其地址连续，循环程序段或子程序段通常被重复执行，因此，指令具有明显的访问局部化特征；

数据在主存一般也是连续存放，特别是数组元素，常常被按序重复访问，因此，数据也具有明显的访问局部化特征。



一、局部性原理

例：

```

1  int sum-array-rows (int a[M][N])
2  {
3      int i, j, sum=0;
4      for (i = 0; i < M; i++)
5          for (j = 0; j < N; j++)
6              sum += a[i][j];
7      return sum;
8  }
    
```

程序段 P2:

```

1  int sum-array-cols (int a[M][N])
2  {
3      int i, j, sum=0;
4      for (j = 0; j < N; j++)
5          for (i = 0; i < M; i++)
6              sum += a[i][j];
7      return sum;
8  }
    
```

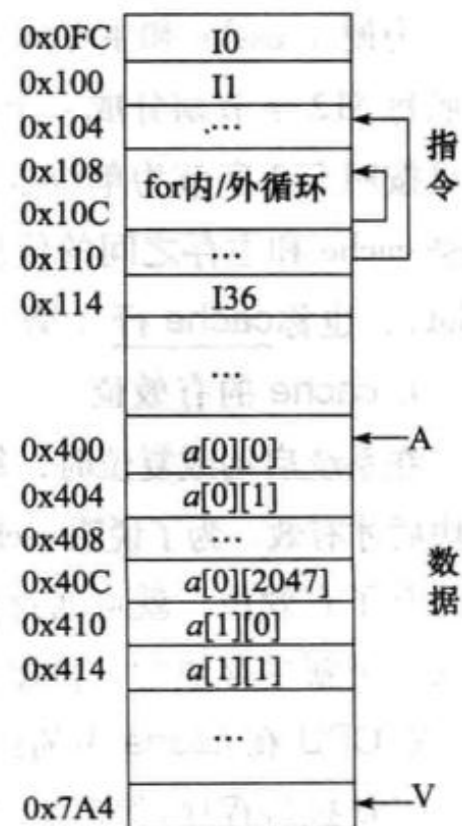


图 6.17 指令和二维数组在主存的存放



一、局部性原理

为了更好地利用程序访问的空间局部性，通常把当前访问单元以及邻近单元作为一个主存块一起调入cache。

这个主存块的大小以及程序对数组元素的访问顺序等都对程序的性能有一定的影响。



二、Cache的工作原理

cache是一种小容量高速缓冲存储器，由快速的SRAM组成，直接制作在CPU芯片内。

cache速度较快，几乎与CPU处于同一个量级。

在CPU和主存之间设置cache,总是把主存中被频繁访问的活跃程序块和数据块复制到cache中。

由于程序访问的局部性，大多数情况下，CPU能直接从cache中取得指令和数据，而不必访问慢速的主存。





二、Cache的工作原理

为便于cache和主存间交换信息，cache和主存空间都被划分为相等的区域。我们把这样的区域称为称为主存块。

CPU执行程序过程中，需要从主存取指令或读写数据时，先检查cache中有没有要访问的信息，若有，就直接在cache中读写，而不用访问主存储器；若没有，再从主存中把该主存块复制到cache中。

二、Cache的工作原理

访存过程如图所示

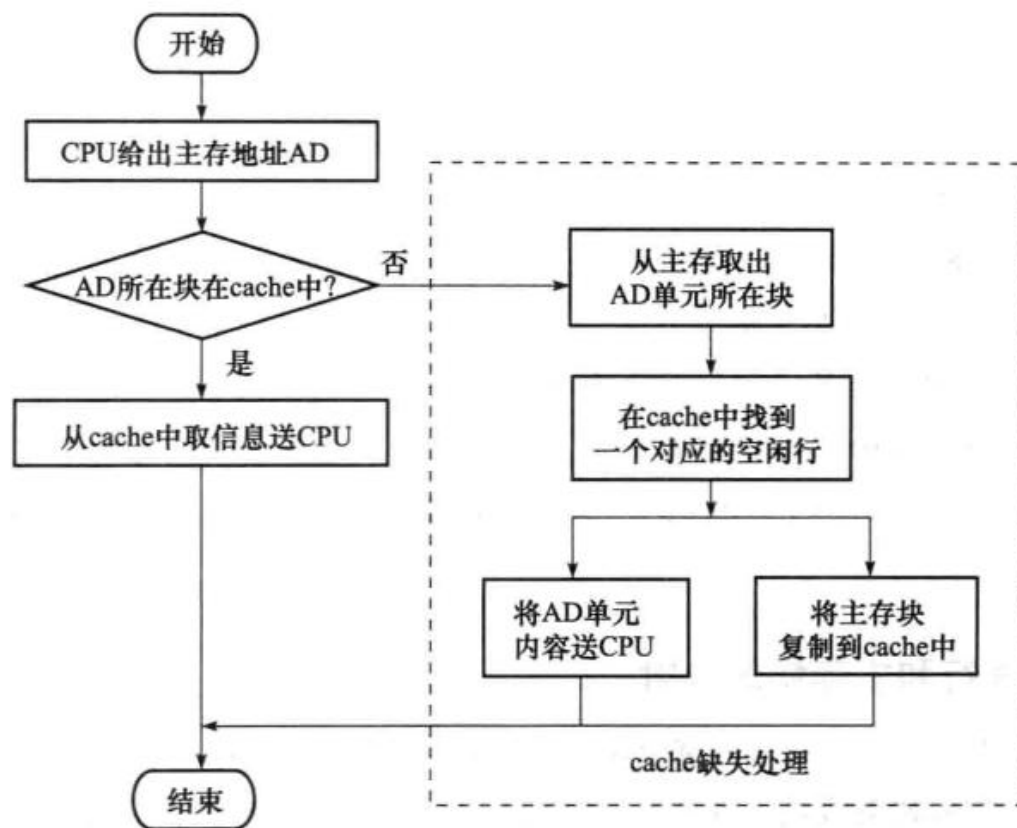


图 6.18 带 cache 的 CPU 的访存操作过程



二、Cache的工作原理

如图所示，整个访存过程包括：判断信息是否在cache中，从cache取信息或从主存取一个主存块到cache等，在对应cache行已满的情况下还要替换cache中的信息。

这些工作要求在一条指令执行过程中完成，且只能由硬件来实现。

cache对程序员来说是透明的，程序员编程时不用考虑信息存放在主存还是在cache。因此，cache位于微体系结构层面，而不位于ISA层面。





cache-主存层次的平均访问时间

如图所示，在访存过程中，需要判断所访问信息是否在cache中。

若CPU访问单元所在的主存块在cache中，则称cache命中，命中的概率称为命中率 p ；若不在cache中，则为不命中。命中时，CPU在cache中直接存取信息，所用的时间开销就是cache访问时间 T_c （命中时间）；缺失时，需要从主存读取一个主存块送cache,并同时所需信息送CPU,用时间开销为主存访问时间 T_m （也称为缺失损失）和cache访问时间 T_c 之和。





cache-主存层次的平均访问时间

则CPU在cache-主存层次的平均访问时间为：

$$T_a = p * T_c + (1 - p) * (T_m + T_c) = T_c + (1 - p) *$$

T_m 。

由于程序访问的局部性特点，cache的命中率可以达到很高，接近于1。因此，平均访问时间 T_a 可接近cache的访问时间 T_c 。





三、Cache和程序性能

显然，程序的性能与程序访问指令和数据所用的时间有很大关系。

指令和数据的访问时间与cache命中率、命中时间和缺失损失有关。

对于给定的计算机系统而言，命中时间和缺失损失是确定的。

因此，指令和数据的访存时间主要由cache命中率决定，而cache命中率则主要由程序的空间局部性和时间局部性决定。





三、Cache和程序性能

因此，为了提高程序的性能，程序员须编写出具有良好访问局部性的程序。

考虑程序的访问局部性通常是考虑数据的访问局部性，数据的访问局部性主要是指数组、结构等类型数据访问时的局部性。

这些数据结构的数据元素访问通常是通过循环语句进行的，因此，合理地处理循环，特别是内循环，对于数据访问局部性来说非常重要。





三、Cache和程序性能

例:

```
#include <stdio.h>
#define M 256
#define N 256
int sumarrayrows(int a[M][N])
{
    int i, j, sum = 0;
    for (i = 0; i < M; i++)
    {
        for (j = 0; j < N; j++)
            sum += a[i][j];
    }
    return sum;
}
```

```
int sumarraycols(int a[M][N])
{
    int i, j, sum = 0;
    for (j = 0; j < N; j++)
    {
        for (i = 0; i < M; i++)
            sum += a[i][j];
    }
    return sum;
}
```



三、Cache和程序性能

```
int main()
{
    int a[M][N], sum;
    sum = sumarrayrows(a);
    sum = sumarraycols(a);
    return 0;
}
```

数组a按照行优先顺序存储

函数sumarrayrows的内循环按行优先顺序访问数组a，具有良好的空间局部性

函数sumarraycols的内循环，按列顺序访问数组a，空间局部性很差，执行效率低。

